

PROGRAMMATION FONCTIONNELLE 2 : TD5

février 2026 — Pierre Rousselin

On rappelle les définitions des listes et des entiers naturels en Rocq, de l'addition des entiers et de la concaténation des listes :

```
Inductive nat : Set := 0 : nat | S : nat -> nat.
Inductive list (A : Type) : Type :=
  nil : list A | cons : A -> list A -> list A.
Fixpoint app {A : Type} (l1 l2 : list A) :=
  match l1 with
  | [] => l2
  | h :: t => h :: (app t l2)
  end.
Fixpoint add (n m : nat) :=
  match n with
  | 0 => m
  | S p => S (add p m)
  end.
```

De plus, on a les notations suivantes :

```
Locate "+". (* Notation "x + y" := (add x y). *)
Print Notation "_ + _". (* level 50, left associativity *)

Locate "++". (* Notation "x ++ y" := app x y *)
Print Notation "_ ++ _". (* level 60, right associativity *)
```

Exercice 1 : Preuves par induction

1. On considère la preuve suivante (comme modèle).

```
Lemma app_nil_r {A : Type} (l : list A) : l ++ [] = l.
Proof.
  induction l as [| h t IHt].
  - simpl.
    reflexivity.
  - simpl.
    rewrite IHt.
    reflexivity.
Qed.
```

Représenter le but au début de chaque sous-preuve et après chaque tactique `simpl` et `rewrite`.

2. Terminer la preuve suivante. Quel est le but et quel est le type de IH?

```
Lemma add_0_r (n : nat) : n + 0 = n.
Proof.
  induction n as [| p IH].
  - simpl.
    reflexivity.
  -
    Qed.
```

3. Quelle est la preuve commune des 4 lemmes suivants?

```
Lemma add_0_l (n : nat) : 0 + n = n.
Lemma add_succ_l (n m : nat) : (S n) + m = S (n + m).
Lemma app_nil_l {A : Type} (l : list A) : [] ++ l = l.
Lemma app_cons_l {A : Type} (t l : list A) (h : A) : (h :: t) ++ l = h :: (t ++ l).
```

4. Voici une « preuve papier très détaillée » de `app_nil_r` :

Démonstration. On raisonne par induction sur l .

- Dans le cas où $l = []$, $l ++ [] = [] ++ [] \stackrel{\text{app_nil-1}}{=} [] = l$.
- Dans le cas où $l = h :: q$, avec q satisfaisant

$$q ++ [] = q, \quad (\text{IH})$$

on a :

$$l ++ [] = (h :: q) ++ [] \stackrel{\text{app_cons-1}}{=} h :: (q ++ []) \stackrel{\text{IH}}{=} h :: q = l. \quad \square$$

5. Donner la « preuve papier très détaillée » de `add_0_r`.
6. Prouver le lemme suivant en Rocq :
`Lemma add_succ_r (n m : nat) : n + S m = S (n + m).`
7. Pour prouver le lemme suivant, vous aurez besoin des lemmes précédents sur l'addition, qu'on utilisera sous forme de règles de réécriture. Par exemple, la tactique `rewrite add_0_r` cherche dans le but un terme de la forme `n + 0` (avec `n` une variable à remplacer) pour le changer en `n`.
`Lemma add_comm (n m : nat) : n + m = m + n.`
8. Écrire sous forme de « preuve papier très détaillée » la preuve de `add_comm`.

--- * ---

Correction de l'exercice 1 : Je manque de temps cette fois-ci. Voici juste les preuves en Rocq :

```

Lemma add_0_r (n : nat) : n + 0 = n.
Proof.
  induction n as [ | p IH ].
  - simpl.
    reflexivity.
  - simpl.
    rewrite IH.
    reflexivity.
Qed.

Lemma add_succ_r (n m : nat) : n + S m = S (n + m).
Proof.
  induction n as [ | p IH ].
  - simpl. reflexivity.
  - simpl. rewrite IH. reflexivity.
Qed.

Lemma add_comm (n m : nat) : n + m = m + n.
Proof.
  induction n as [ | p IH ].
  - simpl. rewrite add_0_r. reflexivity.
  - simpl. rewrite IH. rewrite add_succ_r. reflexivity.
Qed.
```

--- * ---

Exercice 2 : Opérations sur les listes

1. Donner le type complet et la définition de :
 - a) `length`, longueur d'une liste
 - b) `map`, appliquer une fonction à chaque élément d'une liste
 - c) `filter`, garder seulement les éléments d'une liste qui satisfont un prédicat (booléen)
 - d) `all`, dire si tous les éléments d'une liste satisfont un prédicat (booléen)
 - e) `any`, dire si au moins un élément d'une liste satisfait un prédicat (booléen)

- f) `seq n m`, créer la liste des `m` entiers naturels à partir de l'entier `n` (par exemple `seq 2 3` vaut `[2; 3; 4]`).
- Énoncer et prouver un résultat sur la longueur de `map`
 - Énoncer et prouver un résultat sur la longueur de `seq`. Est-ce que votre hypothèse d'induction est suffisamment générale? Si ce n'est pas le cas, on peut utiliser la variante suivante de la tactique induction :

```
induction l as [| h t IH] in m |- *.
```

 Elle commence par généraliser le but avant l'induction proprement dite. On peut aussi utiliser `revert m. avant induction.`

--- * ---

Correction de l'exercice 2 :

```
Fixpoint length {A : Type} (l : list A) :=
  match l with
  | [] => 0
  | _ :: q => S (length q)
end.

Fixpoint map {A B : Type} (f : A -> B) (l : list A) :=
  match l with
  | [] => []
  | h :: q => f h :: map f q
end.

Fixpoint filter {A : Type} (p : A -> bool) (l : list A) :=
  match l with
  | [] => []
  | h :: q => let fq := filter p q in if p h then h :: fq else fq
end.

Fixpoint seq (n m : nat) :=
  match m with
  | 0 => []
  | S p => n :: seq (S n) p
end.

Fixpoint all {A : Type} (p : A -> bool) (l : list A) :=
  match l with
  | [] => true
  | h :: q => if p h then all p q else false
end.

Fixpoint any {A : Type} (p : A -> bool) (l : list A) :=
  match l with
  | [] => false
  | h :: q => if p h then true else any p q
end.

Lemma length_map {A B : Type} (f : A -> B) (l : list A) :
  length (map f l) = length l.
Proof.
  induction l as [| h t IH].
  - simpl. reflexivity.
  - simpl. rewrite IH. reflexivity.
```

Qed.

Lemma length_seq (n m : nat) : length (seq n m) = m.

Proof.

induction m **as** [| p IH] **in** n |- *.

 - **reflexivity**.

 - **simpl. rewrite** IH. **reflexivity**.

Qed.

--- * ---